Berlin Blues BBS

Setup and build guide: cc65 + fujinet-lib, atari800 + FujiNet-PC, Altirra + FujiNet, building `bbs.xex`, customizing

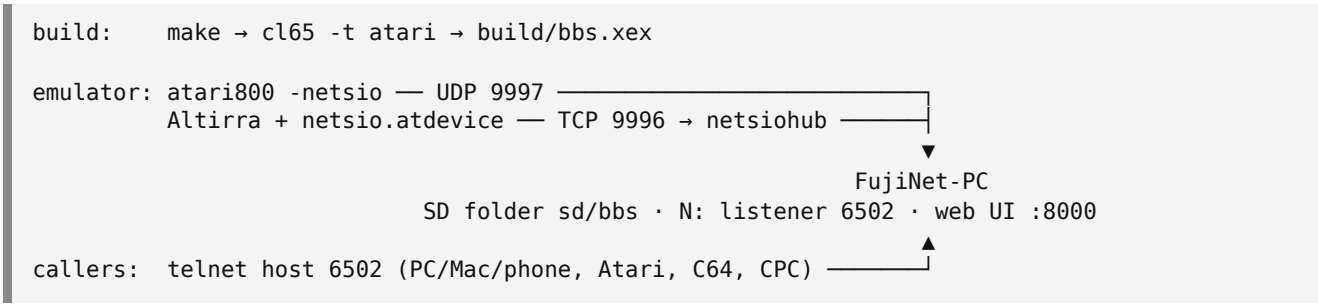
Atari XL/XE + FujiNet, single-line telnet BBS (port 6502). Linux host (tested on Ubuntu). Versions as used: cc65 V2.19 (git), fujinet-lib 5.1.0, atari800 7.2.1, FujiNet-PC v1.6, Altirra 4.40. Program flow: `docs/pap/pap.pdf`. Internals: `docs/TECHNICAL.md`. As of 2026-11-05.

Contents

1. Overview and folder layout
2. Prerequisites
3. cc65 (C compiler)
4. fujinet-lib
5. atari800 with NetSIO
6. FujiNet-PC
7. Altirra + FujiNet (Wine)
8. Atari ROMs
9. Building bbs.xex
10. Running the BBS
11. Customizing
12. Troubleshooting

1. Overview and folder layout

The BBS is a C program compiled with **cc65** against **fujinet-lib** into an Atari executable (`bbs.xex`). It runs on a real Atari with a FujiNet, or on a PC in an emulator. In the emulator setup, the emulated Atari talks to **FujiNet-PC** (the FujiNet firmware built as a Linux program) over **NetSIO**, a network version of the Atari SIO bus. FujiNet-PC provides the network (`N:` device: the TCP listener callers connect to) and the SD card (a folder on the PC).



Two sibling folders. All tools are built into `opt/` inside the project; nothing is installed system-wide (except the distribution packages in section 2).

Folder	Contents
`atari-dev/`	Toolchain and emulators: `src/` (git clones), `opt/cc65`, `opt/fujinet-lib`, `opt/atari800`, `opt/fujinet-pc`, `opt/altirra`; Makefile with `make run / boot / altirra` for a sample program
`blues-bbs/`	The BBS: `src/` (C), `lang/` (texts), `sd/bbs/` (BBS data = SD card), `opt/fujinet-pc/` (its own FujiNet-PC instance), `tools/`, `docs/`

The BBS Makefile finds the toolchain via `DEV ?= ../atari-dev`; with another location: `make DEV=/path/to/atari-dev`.

2. Prerequisites

```
sudo apt install build-essential git make gcc python3 python3-venv cmake \
                 autoconf automake libsdl2-dev unzip telnet
sudo apt install wine          # only for Altirra
sudo apt install gh            # optional: download fujinet-lib releases
```

> **Do not use the distribution's `atari800` package** (Ubuntu ships 5.2.0): it has no NetSIO and cannot talk to FujiNet-PC. Build 7.2.1 from source (section 5).

```
mkdir -p atari-dev/src atari-dev/opt && cd atari-dev/src
git clone https://github.com/cc65/cc65.git
git clone https://github.com/atari800/atari800.git
git clone https://github.com/FujiNetWIFI/fujinet-firmware.git
git clone https://github.com/FujiNetWIFI/fujinet-emulator-bridge.git   # netsiohub + Altirra
device
```

3. cc65 (C compiler)

```
cd atari-dev/src/cc65
make -j$(nproc)
make install PREFIX=$PWD/../../opt/cc65
../../opt/cc65/bin/cl65 --version          # cl65 V2.19 - Git ...
```

The Makefiles set `CC65_HOME=opt/cc65/share/cc65` themselves, so no shell setup is needed.

> **Target `-t atari`, not `-t atarixl`.** fujinet-lib calls the OS SIO vector (`$E459`) directly; with the atarixl runtime the OS ROM is switched out and the call hangs. `-t atari` programs run on every XL/XE; memory is `$2000–$BC20` (~39 KB), atarixl would only give ~4 KB more.

4. fujinet-lib

Use the prebuilt Atari release (5.1.0), not a source build:

```
mkdir -p atari-dev/opt/fujinet-lib && cd atari-dev/opt/fujinet-lib
gh release download -R FujiNetWIFI/fujinet-lib -p "*atari*"     # or download from the GitHub
releases page
unzip fujinet-lib-atari-*.zip
ls fujinet-atari-*.lib fujinet-network.h
```

The Makefiles pick up `fujinet-atari-*.lib` by wildcard and add the folder as include path (`-I opt/fujinet-lib`). Note: `network_init()` is a no-op in 5.x and does not detect a FujiNet.

5. atari800 with NetSIO

```
cd atari-dev/src/atari800
./autogen.sh
./configure --prefix=$PWD/../../opt/atari800 --target=default \
            --with-video=sdl2 --with-sound=sdl2
make -j$(nproc) && make install
../../opt/atari800/bin/atari800 -help | grep -i netsio      # must list -netsio
```

Both projects pass their own config with `-config atari800.cfg`, so `~/.atari800.cfg` is never used. The BBS starts it as:

```
atari800 -config atari800.cfg -no-kbdjoy0 -no-kbdjoy1 -xl -netsio -run build/bbs.xex
```

- `-netsio`: SIO over UDP to FujiNet-PC (default port 9997). Start FujiNet-PC **before** the emulator.
- `-no-kbdjoy0 -no-kbdjoy1`: otherwise W/A/S/D, the keypad and Ctrl act as joysticks and the sysop keys get lost.
- F8 opens atari800's simple monitor; for symbols and source-level debugging use Altirra.

6. FujiNet-PC

```
cd atari-dev/src/fujinet-firmware
./build.sh -y -p ATARI                    # creates a Python venv, builds with cmake
cp -r build/dist ../../opt/fujinet-pc
```

The BBS uses its **own copy**, so the BBS data never mixes with other projects:

```
cp -r atari-dev/opt/fujinet-pc blues-bbs/opt/fujinet-pc
ln -sfn ../../sd blues-bbs/opt/fujinet-pc/SD      # make run does this too
```

- **SD card** = the folder `SD` next to the `fujinet` binary (here a link to `blues-bbs/sd`). The BBS reads and writes `N2:SD:///bbs/...` (three slashes).
- **Config**: `fnconfig.ini`. NetSIO (`[BOIP] enabled=1 host=localhost`) is on by default. Set the `timezone` - the BBS takes date and time from FujiNet (APETIME).
- **Web UI**: `http://localhost:8000` - mount images, settings. It has no login: **never forward port 8000** to the internet.
- **Restarts**: the BBS starts FujiNet-PC through `tools/run-fujinet`, which restarts it after every exit except a deliberate stop - FujiNet-PC crashes (exit 134) when the Atari reboots while the BBS listener is open.

Boot from FujiNet D1 (optional, like the real machine)

For `make boot` in atari-dev: `configenabled=0` and a D1 mount in `fnconfig.ini`, then atari800 is started without `-run` and boots the XEX from FujiNet:

```
[Mount1]
hostslot=1
path=/app.xex
mode=r
```

7. Altirra + FujiNet (Wine)

Altirra is the more exact emulator with a full debugger (symbols, source-level breakpoints). It runs under Wine and reaches FujiNet-PC through a custom device plus the **netsiohub**:

```
Altirra + netsio.atdevice ── TCP 9996 ── netsiohub (Python) ── UDP 9997 ── FujiNet-PC
```

1. Download `Altirra-4.40.zip` from `https://www.virtualdub.org/altirra.html` and unpack it to `atari-dev/opt/altirra/` (portable, no installer).
2. Copy the NetSIO device:
   `cp src/fujinet-emulator-bridge/altirra-custom-device/netsio.atdevice opt/altirra/`
3. Start everything from `atari-dev/`:

```
make altirra        # starts netsiohub (TCP 9996) + FujiNet-PC, then Altirra with
build/app.xex
```

which runs `tools/altirra.sh`: Altirra64.exe `/portable /hardware:800xl /setdevice "custom,path=netsio.atdevice" /debug /run app.xex`. Settings stay in `opt/altirra/Altirra.ini`. The command goes through a `.bat` file because Wine re-quotes arguments and Altirra would reject `/debugcmd:"..."`.

4. The netsiohub alone: `cd src/fujinet-emulator-bridge/fujinet-bridge && python3 -m netsiohub` (log in `build/hub.log`).

Symbols from `.lbl` / `.dbg` load automatically; breakpoints like `bp \`main.c:24\`` work. `make altirra-test` runs a program for 45 s and prints its text screen (debugger script dumps screen RAM).

Running the BBS in Altirra

```
cd blues-bbs && make                                # build/bbs.xex (+ .lbl/.dbg)
cd ../atari-dev && make hub                         # netsiohub on TCP 9996
(cd ../blues-bbs && make fujinet)                   # FujiNet-PC with the BBS SD folder
tools/altirra.sh ../blues-bbs/build/bbs.xex         # Altirra, debugger open
```

Not tested with the BBS so far (only the atari-dev sample). Run only **one** emulator at a time on one FujiNet-PC: netsiohub and atari800 both use UDP 9997 (`make run` stops the hub automatically). Never do this on the machine where the live BBS is running - use a separate box or stop the live BBS first.

8. Atari ROMs

Atari's OS and BASIC ROMs are copyrighted and **not** in the repositories. Two options:

- **Built-in**: delete the `ROM_OS_BB01R2=` line in `atari800.cfg` - atari800 then uses AltirraOS (free replacement). All tests of the atari-dev sample ran that way.
- **Original XL OS**: put your own dump into `rom/ATARIXL.ROM` and set `ROM_OS_BB01R2=/full/path/to/rom/ATARIXL.ROM` in `atari800.cfg` (the live BBS uses this). `rom/` is git-ignored - never commit it.

BASIC is disabled (`DISABLE_BASIC=1`), the machine is an XL/XE with 64 KB, PAL.

9. Building bbs.xex

```
cd blues-bbs
echo 'BBS_LANG = de' > config.mk     # once: German texts (default en); config.mk is not in
git
make                                 # → build/bbs.xex
```

What `make` does

1. `tools/mklang.py` turns `lang/<BBS_LANG>.txt` into `build/lang/<lang>/lang.h` (`L_KEY` macros). It checks placeholders (`%u %s` ...) against `lang/en.txt`, plain ASCII and line length; missing keys fall back to English.
2. `cl65` compiles and links all sources plus fujinet-lib in one step:

```
cl65 -t atari -O -I opt/fujinet-lib -g -I build/lang/de \
     -m build/bbs.map -Ln build/bbs.lbl -Wl --dbgfile,build/bbs.dbg \
     -o build/bbs.xex src/bbs.c src/net.c src/term.c src/sound.c src/store.c src/user.c \
        src/menu.c src/msg.c src/board.c src/pm.c src/sysop.c src/kbd.s fujinet-atari-
5.1.0.lib
```

3. Switching the language (`make BBS_LANG=en`) rebuilds the XEX automatically (`build/lang.cur`).

Output	Use
`build/bbs.xex`	the program - runs in atari800/Altirra, or copy it to the FujiNet SD card
`build/bbs.map`	memory map: segment sizes (watch the ~39 KB limit)
`build/bbs.lbl`, `bbs.dbg`	symbols and source lines for the Altirra debugger

Make targets

Command	Does
`make`	build `build/bbs.xex`
`make run`	stop the live instance, link `SD` → `sd/`, start FujiNet-PC (via run-fujinet) and atari800 with the BBS
`make stop`	stop FujiNet-PC and atari800 of the live instance
`make reset PW=secret`	**deletes all BBS data** (users, stats, boards, mailboxes) and sets the sysop password for the next start
`make sd`	copy the XEX to `sd/bbs.xex`
`make test`	test builds in `build/test/`: port 6503, short timeouts, fast clock wrap, always German
`tools/test_all.sh`	all automated tests on the separate test instance (fresh data)
`make clean`	remove `build/`

Ports and instances

	Live (`make run`)	**Test (`tools/emu.sh`)**
BBS telnet port	6502 (`bbs.cfg`)	6503
NetSIO (UDP)	9997	9998
FujiNet-PC web UI	8000	127.0.0.1:8001
SD folder	`sd/` (real data)	`build/testsd/`
Log	`build/fujinet.log`	`build/fujinet-test.log`

Stop single instances with `tools/inst.sh live|test fujinet|atari800|stop` - never `pkill fujinet`, that would hit both.

10. Running the BBS

a) Emulator (Linux PC) - how the BBS runs online

```
make reset PW=yourpassword     # first time only: fresh data + sysop password
make run                       # atari800 window = sysop console
telnet localhost 6502          # second terminal: call your BBS, log in as SYSOP
```

- **Online**: forward TCP 6502 on the router to the PC (fixed LAN address via DHCP reservation), open the firewall (`sudo ufw allow 6502/tcp`), test from outside (phone on mobile data). Never forward 8000.
- `make run` always restarts FujiNet-PC and atari800 together: a restarted atari800 never links to a FujiNet-PC that is already running.
- **Backup** hourly from cron: `0 * * * * /path/to/blues-bbs/tools/backup.sh` (snapshots of `sd/bbs` in `backup/`, keeps 30; restore: `make stop`, `tar -xzf backup/bbs-<date>.tar.gz -C sd`).
- **Log rotation** hourly: `5 * * * * /path/to/blues-bbs/tools/logrotate.sh` (FujiNet-PC logs ~15 MB/h).

b) Real hardware (not tested yet)

1. Atari 800XL/130XE (64 KB) + FujiNet with SD card, connected to the WLAN.
2. Copy `build/bbs.xex` to the card, and `sd/bbs/` as `/bbs/` (menus, texts, `bbs.cfg`). Put the initial sysop password into `/bbs/sysop.txt` (one line) - the BBS deletes it after the first start.
3. Boot the Atari with FujiNet, select the SD host, boot `bbs.xex`. FujiNet boots XEX files directly (no ATR needed).
4. Forward TCP 6502 to the FujiNet's IP address. Back up by copying `/bbs/` from the card now and then.

First start

The BBS waits for the SD card (retries every 5 s), loads `menus.txt`, reads `bbs.cfg`, opens `users.dat` (creates user `SYSOP` with the password from `sysop.txt` if it is new) and opens the listener (retries every 10 s). Then it shows "waiting for call".

Sysop keys on the Atari

Key	Does
A	sysop present / away - callers can page you only when present (paging rings 30 s, then a PM to the sysop)
C	chat with the caller (when paged or any time during a call); ESC ends it
K	kick the caller
L	local login on the Atari keyboard and screen (while no one is calling)
Q	shut down cleanly (after the current call)

11. Customizing

`bbs/` below means `sd/bbs/` in the emulator and `/bbs/` on the FujiNet card. Files are read at boot - **restart the BBS after a change**. All texts plain ASCII (umlauts as ae/oe/ue/ss), max. 40 columns for Atari and C64 callers.

Without rebuilding (SD card files)

What	File	Notes
Port	`bbs/bbs.cfg`	`port = 6502`; `key = value`, `#` comments; 8000 is refused; router + firewall must match
Welcome banner	`bbs/texts/banner.txt`	shown after the terminal choice, before login
Help/info texts	`bbs/texts/*.txt`	linked from menus with type `T`
Menus and boards	`bbs/menus.txt`	see below; max. 48 items, 16 boards
First sysop password	`bbs/sysop.txt`	used once when `users.dat` is created, then deleted

`menus.txt` format

```
# <menu> <key> <type> <arg> <level> <label>
0 1 M 1  0 Hilfe & Info's          # main menu, key 1 → submenu 1
0 2 B 1  0 Termine & Treffen        # board 1
0 7 F new 0 Neue Msg lesen          # function
1 1 T kurz.txt 0 Kurz-Anleitung     # text file bbs/texts/kurz.txt
8 9 F approve 5 Freischalten (Sysop)  # level 5 = sysop only
```

Field	Meaning
menu	menu number; 0 = main menu. A submenu's title is the label of the item leading to it.
key	one character the caller types (digits, like 1987); chains like `*42` work, `*` = main menu, RETURN = back, 0 = logoff
type	`M` submenu (arg = menu no.), `B` board (arg = 1-16), `T` text file, `F` function
functions	`status users passwd approve new pmwrite pmread sysop`
level	minimum level to see/use: 0 guest, 1 new (read only, waits for approval), 2 member, 5 sysop
label	max. 22 characters

Program texts and languages (rebuild)

- `lang/en.txt` is the master list (every key), `lang/de.txt` the German version: `KEY = "text"`, `\n` = new line, same placeholders in the same order.
- New language: copy `en.txt` to `lang/xx.txt`, translate, `make BBS_LANG=xx` (or set it in `config.mk`).
- Source code contains no caller or console texts - always add a key to both files and use `L_KEY`.

Compile-time settings (rebuild)

Constant	Default	Where	Meaning
`IDLE_SECONDS`	300	`src/term.h`	logged in: hang up after 5 min without a key
`LOGIN_IDLE`	60	`src/term.h`	before login: 1 min without a key
`LOGIN_TOTAL`	300	`src/term.h`	before login: 5 min per call at most (against bots)
`PAGE_SECONDS`	30	`src/sysop.c`	how long a page rings the sysop
`BBS_PORT`	6502	`src/net.c`	default port if `bbs.cfg` has none (`-DBBS_PORT=n`)
`MAX_BOARDS`	16	`src/board.h`	number of boards
`NAME_LEN`	15	`src/user.h`	max. user name length (changes the `users.dat` format!)

Online (as sysop)

Menu 8 → 4 changes your password, menu 8 → 9 approves new users (level 1 → 2) so they can write.

12. Troubleshooting

Symptom	Cause / fix
atari800 hangs on boot, or "No FujiNet found"	FujiNet-PC not running or NetSIO link down: start FujiNet-PC first; `make run` restarts both
`atari800: unknown option -netsio`	the distribution's atari800 is used - use `opt/atari800/bin/atari800`
Program hangs at the first FujiNet call	built with `-t atarixl` - use `-t atari`
Sysop keys W/A/S/D/Ctrl don't work	missing `-no-kbdjoy0 -no-kbdjoy1`
"waiting for SD" forever	`SD` link missing in `opt/fujinet-pc/` or no `bbs/menus.txt` on it
"retry listen" repeats	port in use or FujiNet-PC restarting - check `build/fujinet.log`
Altirra: "open failed"	netsiohub not running: `ss -ltn | grep 9996`, log `build/hub.log`
Altirra: *Cannot open file "/debugcmd:..."*	start it via `tools/altirra.sh` (.bat), not directly through Wine
Disk filling up	FujiNet-PC log: install the `tools/logrotate.sh` cron job
Wrong date/time	`timezone` in `fnconfig.ini`; time comes from FujiNet (SNTP)

Atari clock note: the Atari's tick counter (RTCLOK) wraps every ~3.9 days; all time math in the BBS uses `ELAPSED()` (`src/ticks.h`), so the wrap is harmless. Dates come from FujiNet, not from RTCLOK.